Cholesky: el problema que vamos a resolver

Queremos factorizar una matriz real A, simétrica y definida positiva, como el producto de una matriz triangular inferior L y su transpuesta:

$$A = LL^T$$

Por encima de la diagonal de L todos los elementos son cero. Elegimos su diagonal positiva; con esa condición el factor es único. Esta factorización permite resolver Ax = b mediante los sistemas triangulares Ly = b y $L^T x = y$.

Condiciones de entrada

$$A = A^T, \qquad x^T A x > 0 \quad \text{para todo } x \neq 0$$

La primera condición es simetría; la segunda es definición positiva. En aritmética exacta garantizan que los pivotes de Cholesky sean positivos, sin necesidad de intercambiar filas. El código comprueba los pivotes para detectar entradas o resultados inválidos.

De dónde salen las dependencias

Al desarrollar el producto matricial, con índices desde cero, obtenemos:

$$a_{ij} = \sum_{k=0}^{\min(i,j)} l_{ik} l_{jk}$$

Una columna nueva utiliza contribuciones de las anteriores. Left-looking y right-looking cambian cuándo se aplican esas contribuciones, no el problema matemático. Ambas variantes escalares realizan aproximadamente $n^3/3$ operaciones de punto flotante.

Las tres decisiones del trabajo

Primera: qué variante conviene estudiar y qué muestran sus tiempos. **Segunda:** cuándo el ahorro de cálculo justifica usar más procesos. **Tercera:** cómo repartir los datos para mantener trabajo útil en esos procesos.

Todo el cálculo, la generación de matrices y la futura implementación MPI se realizan en C. Python procesa los CSV y genera estadísticas y gráficos. En este informe se distingue entre mediciones ejecutadas, conteos analíticos y experimentos MPI todavía pendientes.

[1] Ballard et al. (2009), Communication-optimal Parallel and Sequential Cholesky Decomposition.

Left-looking y right-looking: ¿dónde está el paralelismo?

Left-looking: actualizar la columna que necesitamos

Al llegar a la columna j, recuperamos los aportes de las columnas ya calculadas. Primero obtenemos el pivote y luego los elementos debajo de él:

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=0}^{j-1} l_{jk}^2}$$

$$l_{ij} = \frac{a_{ij} - \sum_{k=0}^{j-1} l_{ik} l_{jk}}{l_{jj}}, \qquad i > j$$

También permite paralelismo: conocido el pivote y disponibles los datos anteriores, distintos elementos l_{ij} pueden calcularse simultáneamente. Incluso una suma interna puede repartirse si luego se combinan correctamente sus resultados. En memoria distribuida debemos decidir dónde están esos datos y cuánto cuesta reunirlos.

Right-looking: actualizar la región que todavía falta

Usamos una matriz de trabajo S, inicialmente igual a A. Al llegar a j, las contribuciones anteriores ya se aplicaron:

$$l_{jj} = \sqrt{s_{jj}}, \qquad l_{ij} = s_{ij}/l_{jj} \quad (i > j)$$

Luego la columna calculada actualiza la submatriz restante:

$$s_{ik} \leftarrow s_{ik} - l_{ij} l_{kj}, \qquad i \geq k > j$$

El paralelismo aparece en esas actualizaciones: cada pareja (i, k) modifica una posición diferente y lee una columna ya terminada. Las actualizaciones de la siguiente columna deben completarse antes de factorizarla.

Ambas variantes tienen dependencias entre columnas y admiten versiones por bloques. En right-looking nos interesa repartir las actualizaciones; no ejecutar arbitrariamente todas las columnas a la vez. [1]

Experimento 1: comparar las dos variantes en C

La pregunta y el experimento

¿Right-looking es más rápido que left-looking en nuestra implementación secuencial? Implementamos ambas variantes escalares en el mismo archivo C, con double, almacenamiento por filas y el mismo compilador. No usamos BLAS ni MPI en esta comparación.

Matrices iguales para las dos variantes

El generador C, con semilla 2026, crea valores simétricos fuera de la diagonal en [−1, 1). La diagonal se define como:

$$a_{ii} = 1 + \sum_{j \neq i} |a_{ij}|$$

Esto produce una matriz simétrica con diagonal positiva y estrictamente dominante; por tanto es definida positiva. Cada llamada recibe una copia nueva de la misma matriz. Se exige un residuo relativo de reconstrucción no mayor que 10^{-12}.

Condiciones de medición

WSL / Ubuntu 26.04, AMD Ryzen 5 2600, GCC 15.2.0 y opciones -O3 -std=c11 -Wall -Wextra -Wpedantic. Ambas variantes se fijan a la misma CPU lógica. Para cada tamaño hacemos un calentamiento por variante y siete repeticiones, alternando cuál se ejecuta primero.

El tiempo incluye solo la factorización. Generación, copia y residuo quedan fuera. Conservamos las observaciones individuales y calculamos medianas y cuartiles inclusivos. La copia y validación siguen el mismo protocolo para ambas variantes, pero no garantizan un estado idéntico de caché.

n	Left (ms)	Right (ms)	Right / Left
64	0.0238	0.0226	0.95
128	0.1999	0.2680	1.34
256	1.8608	5.9010	3.17
512	16.2481	130.1355	8.01
1024	137.6874	1109.6237	8.06

Medianas de siete repeticiones por variante y tamaño. Un cociente Right / Left mayor que 1 indica que right-looking tarda más. No es un speedup MPI. Datos nuevos: resultados/left_right_20260921/.

Qué dicen los datos y qué no demuestran

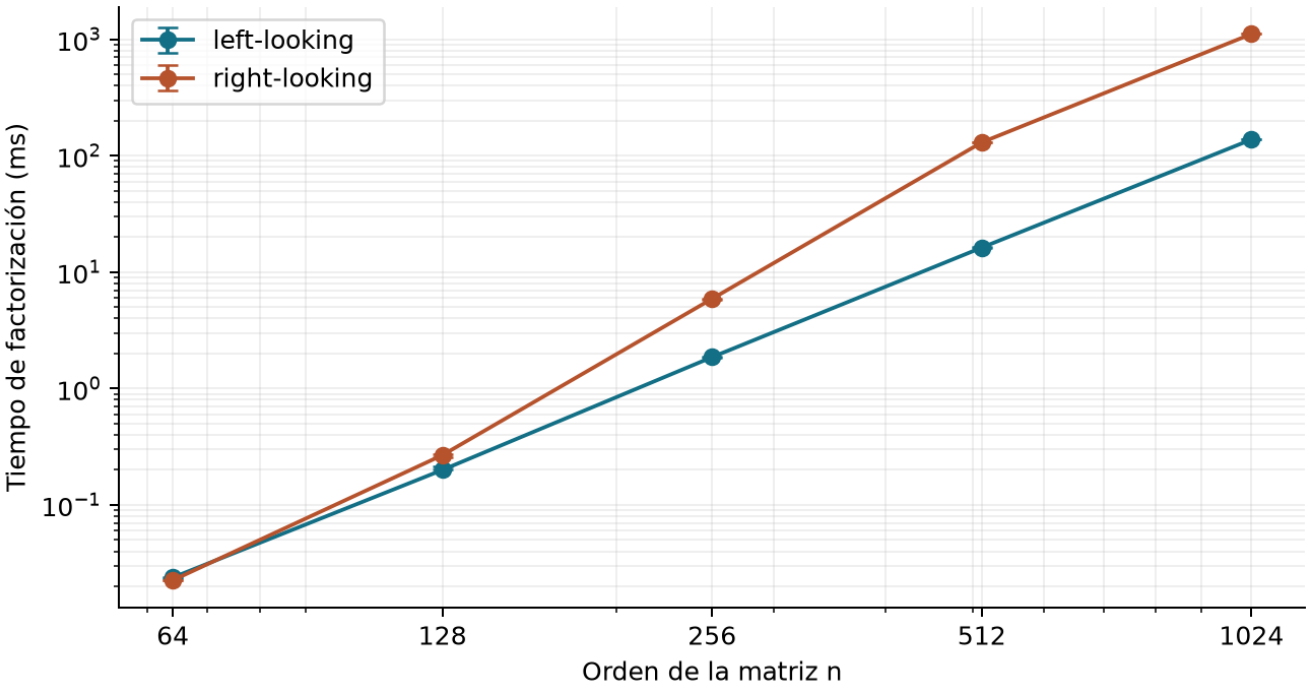


Figura 1. Comparación secuencial. Ambos ejes son logarítmicos. Los puntos indican la mediana y las barras abarcan Q1 a Q3; algunas barras son pequeñas a esta escala.

Resultado observado

En esta primera campaña, right-looking tiene una mediana ligeramente menor para n = 64. Left-looking es más rápido en los otros cuatro tamaños ensayados. Para n = 1024, right-looking tarda aproximadamente 8,06 veces más. Por lo tanto, **estos resultados no justifican elegir la versión original de right-looking por velocidad secuencial**.

Una explicación que debemos investigar

El código de left-looking calcula productos escalares recorriendo filas contiguas. El de right-looking relee elementos de una columna con saltos de n posiciones. La inspección del código muestra esa diferencia de acceso; el experimento no aísla cuánto explica del tiempo. Haría falta variar almacenamiento o implementar bloques para comprobarlo.

La conclusión corresponde a estas versiones escalares y a este equipo. No prueba que todas las implementaciones de left-looking superen a todas las de right-looking, ni que MPI vaya a invertir el resultado.

La pregunta siguiente

¿Qué pasaría si copiamos la columna activa a un vector contiguo antes de reutilizarla? Esa pregunta abre el experimento 2. Antes de agregar comunicación con MPI, comprobamos si un cambio pequeño en los accesos a datos mejora el cálculo local.

Experimento 2: cambiar el acceso, conservar el algoritmo

Hipótesis

El kernel right-looking original recorre la matriz por filas, pero lee la columna activa con saltos. Proponemos copiar esa columna a un vector contiguo y reutilizarlo durante la actualización. La hipótesis es que el ahorro en esas lecturas repetidas compense el costo de copiar.

Versión	Lectura dentro del bucle sobre k
Right original	a[k*n+j]: al aumentar k, la dirección avanza n valores.
Right + vector	column[k]: al aumentar k, la dirección avanza un valor.

$$c_k \leftarrow l_{kj}, \qquad s_{ik} \leftarrow s_{ik} - l_{ij} c_k$$

Mantenemos los bucles en el orden j, i, k, los mismos destinos y las mismas multiplicaciones y restas. El pivote y el cálculo de la columna no cambian. Esto sigue siendo Cholesky right-looking; no es aún una versión por bloques ni una implementación MPI.

Costo de la modificación

$$\text{memoria extra} = 8n \text{ bytes}, \qquad \text{copias} = \frac{n(n-1)}{2}$$

El vector reserva n valores double y se reutiliza. Para n = 1024 ocupa 8 KiB. El total de valores copiados crece como n^2, mientras que las actualizaciones mantienen su trabajo cúbico. Reservamos el vector antes del cronómetro, pero **todas las copias de columnas se incluyen en el tiempo medido**.

Comparación controlada

Volvemos a medir left, right original y right con vector en la misma campaña, sobre las mismas matrices y la misma CPU lógica. Hacemos un calentamiento por variante y doce repeticiones: las seis permutaciones del orden de ejecución se usan dos veces. Así cada variante aparece igual cantidad de veces en cada posición.

Conservamos el equipo, GCC 15.2.0 y las opciones -O3 del experimento anterior. Medimos n = 64, 128, 256, 512, 1000 y 1024; n = 1000 evita limitar la muestra a potencias de dos. Las reservas, la copia de entrada y la validación quedan fuera del tiempo en las tres variantes.

Resultados: ¿compensa copiar la columna?

n	Left (ms)	Right (ms)	Vector (ms)	Right / vector	Left / vector
64	0.0239	0.0230	0.0202	1.14	1.19
128	0.2004	0.2570	0.1725	1.49	1.16
256	1.8380	5.8159	1.3914	4.18	1.32
512	18.0215	185.6965	12.3247	15.07	1.46
1000	131.6738	95.7353	59.8545	1.60	2.20
1024	144.7909	1159.0888	76.9183	15.07	1.88

Medianas de doce repeticiones por variante y tamaño. Un cociente mayor que 1 favorece a right con vector. Cada comparación usa exclusivamente observaciones de esta segunda campaña; no mezclamos tiempos nuevos con los del experimento 1.

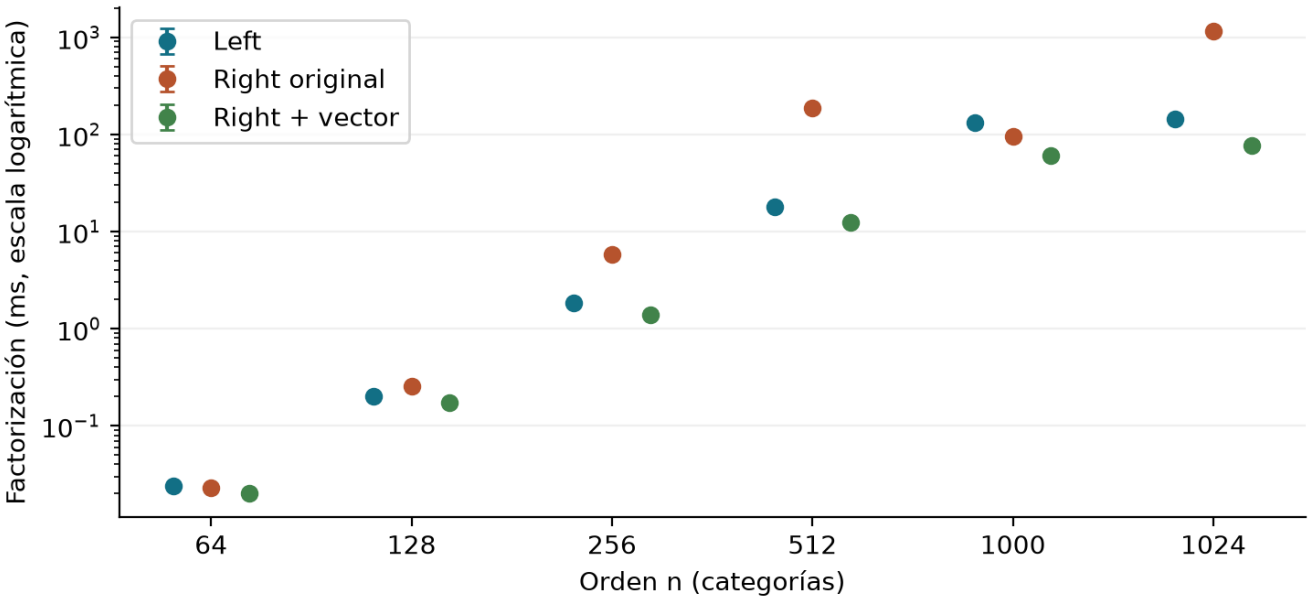


Figura 2. Medianas y barras Q1–Q3. El eje vertical es logarítmico; el horizontal muestra tamaños como categorías. Las barras representan dispersión, no intervalos de confianza.

El resultado que cambia nuestra decisión

La variante con vector obtiene menor mediana en los seis tamaños probados. Para n = 1024, pasa de 1159,09 ms a 76,92 ms frente a right original: una mejora de 15,07 veces. En la misma campaña supera a left, que tarda 144,79 ms, por un factor de 1,88.

La comparación conserva y contabiliza el costo del empaquetado. El trabajo adicional de copia se compensa en estas condiciones. Son mediciones secuenciales locales; todavía no representan un speedup MPI.

Interpretación y decisión después del experimento

¿Se conserva la corrección?

Sí en los casos ensayados. La variante con vector produce el factor esperado para el ejemplo conocido y supera quince matrices generadas de distintos tamaños y semillas. Se compara su factor con right original, se comprueban pivotes inválidos y se prueba que no lea zonas del vector todavía sin inicializar ni escriba fuera de él.

Las pruebas con AddressSanitizer y UndefinedBehaviorSanitizer pasaron. En la campaña se aceptaron 216 observaciones: seis tamaños, tres variantes y doce repeticiones. Todas cumplieron el criterio de residuo relativo no mayor que 10^{-12}; el máximo registrado fue aproximadamente $1{,}21 \times 10^{-15}$.

Qué respalda y qué no aísla esta evidencia

Mantener el algoritmo y cambiar el acceso a la columna produjo una reducción clara del tiempo. Esto respalda que la organización de esos accesos era una limitación importante de esta implementación. Sin contadores de hardware ni análisis adicional no atribuimos un porcentaje específico a fallos de caché: también pueden cambiar la vectorización y otras decisiones del compilador.

Hay una señal útil: right original tarda 95,74 ms para n = 1000 y 1159,09 ms para n = 1024, aunque los tamaños son cercanos. La nueva versión tarda 59,85 y 76,92 ms. La sensibilidad al tamaño se reduce, pero no afirmamos haber identificado el mecanismo exacto que causó el salto.

Qué conservamos

Adoptamos right-looking con vector como referencia local para el siguiente experimento MPI. Ahora la decisión combina una organización clara para repartir actualizaciones con una mejora secuencial medida. Mantenemos left y right original como referencias y preservamos los resultados anteriores: pertenecen a implementaciones distintas.

No concluimos que right-looking sea universalmente superior. La selección vale para estas versiones, matrices y máquina, y debe revisarse en el clúster. Para evaluar MPI volveremos a medir la mejor referencia secuencial en el mismo entorno.

La pregunta siguiente

¿La ganancia de repartir estas actualizaciones alcanza para pagar el envío de columnas? Esa será la prueba MPI. Un kernel local más rápido puede hacer que necesitemos matrices más grandes para que la comunicación se amortice; mejorar lo secuencial no garantiza más speedup paralelo.

Trazabilidad: src/compare_memory.c, scripts/comparar_memoria.sh y resultados/memoria_20260921/. Python produce resumen.csv y memoria.png a partir de los CSV individuales.

La razón para estudiar right-looking con MPI

Partimos de una propiedad fija de las columnas

Supongamos que cada columna tiene un proceso propietario. Queremos que los datos permanezcan en ese proceso y que el trabajo importante pueda repartirse sin migrar continuamente la matriz.

Con ese reparto	Left-looking	Right-looking
Columna j	Su dueño necesita aportes de columnas anteriores que pueden estar en otros procesos.	Su dueño encuentra la columna ya actualizada, calcula el pivote y los valores inferiores.
Trabajo repartido	Para repartir el cálculo de esa columna habría que distribuir sumas o filas y reunir resultados, o usar otra planificación.	La columna calculada se comunica; cada proceso actualiza las columnas futuras de las que ya es dueño.
Dependencias	Los aportes y el pivote deben estar disponibles.	La columna debe estar lista antes de actualizar; las actualizaciones necesarias preceden al siguiente pivote.

La tabla describe una primera organización sencilla con columnas de dueño fijo. No afirma que todas las implementaciones left-looking necesiten esa misma comunicación ni que no puedan ser competitivas.

Dónde encontramos trabajo independiente

Con right-looking, terminada la columna j, dos procesos pueden actualizar destinos diferentes usando los mismos valores recibidos. La región activa contiene muchas actualizaciones, mientras que el pivote sigue siendo parte de la cadena de dependencias. Esta secuencia tiene una traducción directa a MPI: propietario → difusión de columna → actualizaciones locales. [2]

Decisión defendible en el informe

Elegimos right-looking con vector para la primera implementación MPI por su correspondencia con un reparto estático de columnas, sus actualizaciones independientes y la mejora local medida en el experimento 2. Conservaremos left y el mejor secuencial disponible al evaluar la utilidad real de MPI.

La ventaja secuencial observada no demuestra escalabilidad distribuida. La adopción de MPI por rendimiento queda condicionada a medir su costo completo de factorización, incluidas comunicaciones y esperas.

[2] Netlib, Cholesky Factorization: panel y actualización distribuida right-looking.

¿Cuándo conviene usar más procesos?

Repartir trabajo ahorra cálculo por proceso, pero agrega comunicación, coordinación y posibles esperas. Un modelo orientativo, sin solapamiento, es:

$$T_P \approx T_{\text{dep}} + \frac{T_{\text{upd}}}{P} + T_{\text{comm}} + T_{\text{espera}}$$

Tdep representa trabajo dependiente no repartido; Tupd, las actualizaciones repartibles. En este modelo, usar P procesos compensa frente al mismo algoritmo secuencial cuando:

$$T_{\text{upd}}\left(1 - \frac{1}{P}\right) > T_{\text{comm}} + T_{\text{espera}}$$

Para un mensaje de m bytes, el modelo α + βm distingue el costo de iniciar el envío y el costo por volumen. Un broadcast depende además de P y de la implementación MPI; no lo deduciremos únicamente de un ping-pong. [1]

La comparación que realmente decide

$$T_{\text{base}}(n) = \min(T_{L,1}(n), T_{R,1}(n), T_{R_v,1}(n))$$

$$S_{\text{util}}(n, P) = \frac{T_{\text{base}}(n)}{T_{R_v,P}(n)}$$

Rv indica right con vector. Conviene MPI si reduce de forma repetible el tiempo frente a esa base. También compararemos el mismo kernel MPI con P = 1 y P mayor para aislar su escalabilidad. Si importa el tiempo completo de una tarea, debemos sumar arranque y distribución inicial.

Para n = 1024, el experimento 2 deja una referencia local de 76,92 ms. Una versión MPI tendría que mejorar ese tiempo total de factorización en condiciones comparables. No usaremos los 1159,09 ms de right original para presentar un speedup artificialmente favorable.

Cómo obtendremos el umbral

Para cada n probaremos P = 1, 2, 3, 4 y otros valores permitidos, con varias ejecuciones. Elegiremos el menor tiempo estable; si la diferencia es pequeña frente a la dispersión, preferiremos menos recursos. El primer n favorable será un cruce observado para esa configuración, no una constante universal.

Pendiente: aún no ejecutamos MPI en esta etapa ni tenemos resultados entre nodos. WSL no mide la red del clúster. Separaremos ensayos dentro de un nodo y entre nodos, manteniendo explícitos procesos, núcleos y afinidad.

Repartos estáticos: contiguo, cíclico y por bloques

En un reparto estático, el propietario de cada columna se fija antes de factorizar. **Round-robin por columnas es exactamente el reparto cíclico**: se asignan por turnos a los procesos 0, 1, …, P−1 y se repite.

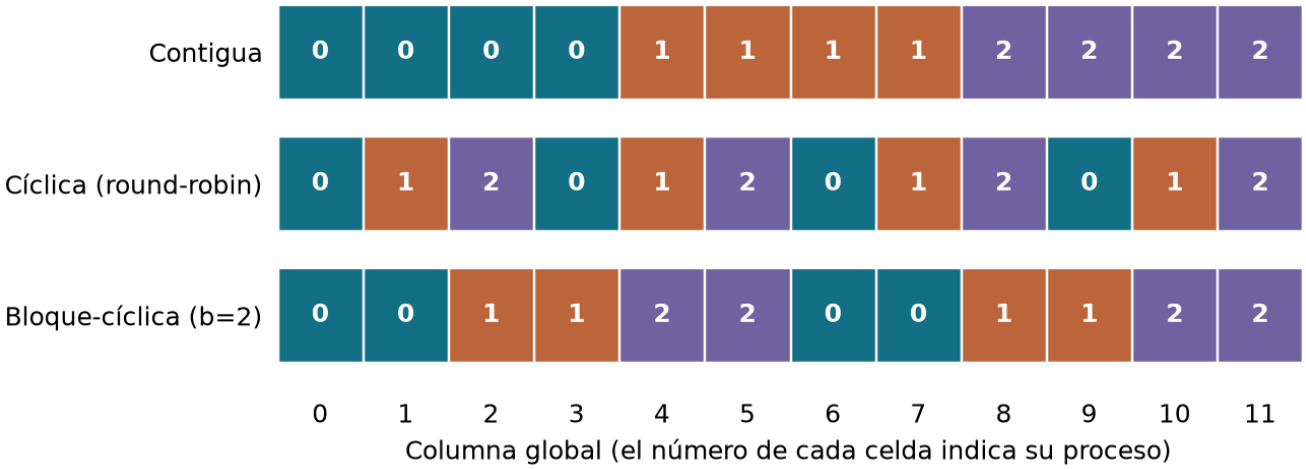


Figura 3. Ejemplo de asignación con n = 12 y P = 3. Los colores y números indican propietarios, no tiempos de ejecución.

Contiguo: un intervalo de columnas por proceso

P0 recibe 0–3, P1 recibe 4–7 y P2 recibe 8–11. La asignación es sencilla. El problema aparece cuando termina el primer intervalo: P0 deja de tener columnas futuras que actualizar mientras otros procesos siguen trabajando. Igual cantidad de columnas no implica igual cantidad de operaciones.

Cíclico: columnas alternadas

$$\text{owner}(j) = j \bmod P$$

P0 recibe 0, 3, 6 y 9; P1 recibe 1, 4, 7 y 10; P2 recibe 2, 5, 8 y 11. Al reducirse la región activa, sus columnas quedan repartidas entre varios procesos durante más etapas. No elimina el desequilibrio ni la escasez de trabajo al final. [3]

Bloque-cíclico: repartir grupos de columnas

$$\text{owner}(j) = \lfloor j/b \rfloor \bmod P$$

Agrupa b columnas consecutivas antes de pasar al siguiente proceso. Con b = 1 recuperamos el cíclico. Con bloques mayores se busca combinar reparto y operaciones locales más grandes; el tamaño b se elegirá midiendo. Cambiar el reparto por sí solo no convierte el kernel escalar en BLAS-3.

[3] ScaLAPACK Users' Guide, Two-dimensional Block-Cyclic Distribution.

Por qué el cíclico es nuestra primera hipótesis

Contemos el trabajo, sin inventar tiempos

En el kernel escalar right-looking, una columna global k recibe una actualización por cada columna anterior j. Cada una modifica sus n−k elementos inferiores. El número de actualizaciones de esa columna durante toda la factorización es:

$$U(k) = k(n-k), \qquad k = 0, \ldots, n-1$$

Sumando U(k) según los propietarios de la figura anterior obtenemos este conteo exacto para n = 12 y P = 3:

Reparto	P0	P1	P2	Máx. / media
Contigua	58	138	90	1.45
Cíclica (round-robin)	90	98	98	1.03
Bloque-cíclica (b=2)	82	106	98	1.11

Se cuentan restas de productos sobre elementos, no pivotes, divisiones, bytes ni segundos. Un valor Máx./media cercano a 1 indica mejor equilibrio de este trabajo acumulado. No representa una medición de rendimiento ni captura todas las esperas entre etapas.

Qué respalda este ejemplo

El cíclico distribuye mejor las actualizaciones que el contiguo en este caso. Para matrices densas y procesos similares, eso motiva comenzar con **propiedad estática cíclica**. Después compararemos bloque-cíclico cuando agrupemos operaciones. La decisión final dependerá del tiempo y de la comunicación medidos, no solo del conteo.

Distribución contigua no significa memoria contigua

El reparto indica qué índices globales pertenecen a un proceso. El almacenamiento local indica cómo los guarda en su RAM. Un proceso con columnas cíclicas puede empaquetarlas en un arreglo local contiguo; no necesita reservar los huecos de las columnas ajenas. [5]

Cuándo asignamos las columnas

En el esquema estático las asignamos al inicio. Durante cada etapa se comunica la columna lista y se trabaja sobre datos locales. No esperamos a un número de columna mágico para empezar a delegar. Cerca del final hay menos destinos independientes; reducir procesos o migrar datos solo convendría si su ahorro supera ese costo adicional.

[5] ScaLAPACK Users' Guide, Local Storage Scheme and Block-Cyclic Mapping.

Reparto dinámico y criterio para continuar

Qué decide una distribución dinámica

Un planificador asigna tareas durante la ejecución según qué procesos están libres. En Cholesky no puede entregar cualquier columna sin más: solo puede despachar tareas cuyas dependencias estén resueltas. Una tarea podría actualizar un bloque usando un panel ya calculado. [4]

Si el proceso elegido no tiene los datos, debe recibirlos o acceder a ellos. Además, dos tareas que modifican el mismo bloque deben respetar su dependencia o combinar sus resultados de forma segura. Por eso distinguimos la propiedad de datos de la planificación de tareas: es posible fijar una y hacer dinámica la otra.

Cuándo podría ayudar y qué cuesta

Aspecto	Estático cíclico	Dinámico
Asignación	Se conoce de antemano; no consulta una cola por tarea.	Se decide al quedar libre un trabajador y estar lista una tarea.
Desigualdad entre procesos	Un proceso lento puede retrasar etapas.	Puede asignar más tareas a trabajadores rápidos.
Datos y coordinación	Propiedad fija; comunicación del panel predecible.	Colas, mensajes, dependencias y posibles transferencias de datos.
Granularidad	Columnas o grupos con tamaño fijado.	Tareas pequeñas equilibran mejor, pero aumentan coordinación; tareas grandes reducen ese costo.

No podemos descartar lo dinámico como siempre más lento: un planificador por tareas puede ser eficaz. Para nuestro primer MPI de matrices densas en recursos similares, el cíclico estático ofrece un punto de partida más sencillo. Lo dinámico se justificaría si medimos esperas importantes por heterogeneidad o trabajo irregular que compensen su costo.

El siguiente experimento, en orden

1. Conservar right con vector como referencia local, junto con left y right original.
2. Implementar MPI con propiedad estática, comparando cíclico y contiguo con el mismo kernel.
3. Medir por separado P y n dentro de un nodo y entre nodos; registrar tiempo total de factorización, cómputo por proceso y espera en MPI.
4. Incorporar bloques o planificación dinámica solo si los resultados explican qué problema resolverían.

La medición paralela usará el máximo de los tiempos locales con inicio coordinado; excluirá validación y mantendrá el mismo criterio en todas las variantes. El tiempo dentro de MPI puede incluir espera, no solo transferencia. No se presentan tiempos de MPI ni de distribuciones como si ya se hubieran medido.

[4] Dorris et al., Task-Based Cholesky Decomposition: tareas listas y planificación dinámica.